RampWise

Accessible Campus Route Planner

Assignment 1: Client-side web application

Name and ID: [your name and ID]

Assigned idea: Accessible Campus Route Planner

Extra rule: [your instructor's rule]

GitHub: [repo link] Live site: [GitHub Pages link]

1. User and problem

My user is a student or visitor who has trouble moving around campus, for example someone in a wheelchair, on crutches, or pushing a stroller. Normal campus maps only show that two buildings are connected. They do not show steps, steep slopes, shelter from rain, or lifts that are not working, so people find out a route is blocked only after they get there.

The workflow in my app is: choose where you are and where you want to go, pick your needs (step-free, covered, avoid steep slopes) and your pace, then read the routes that match with the walking time. If a route is blocked, the user can report it and it shows as closed.

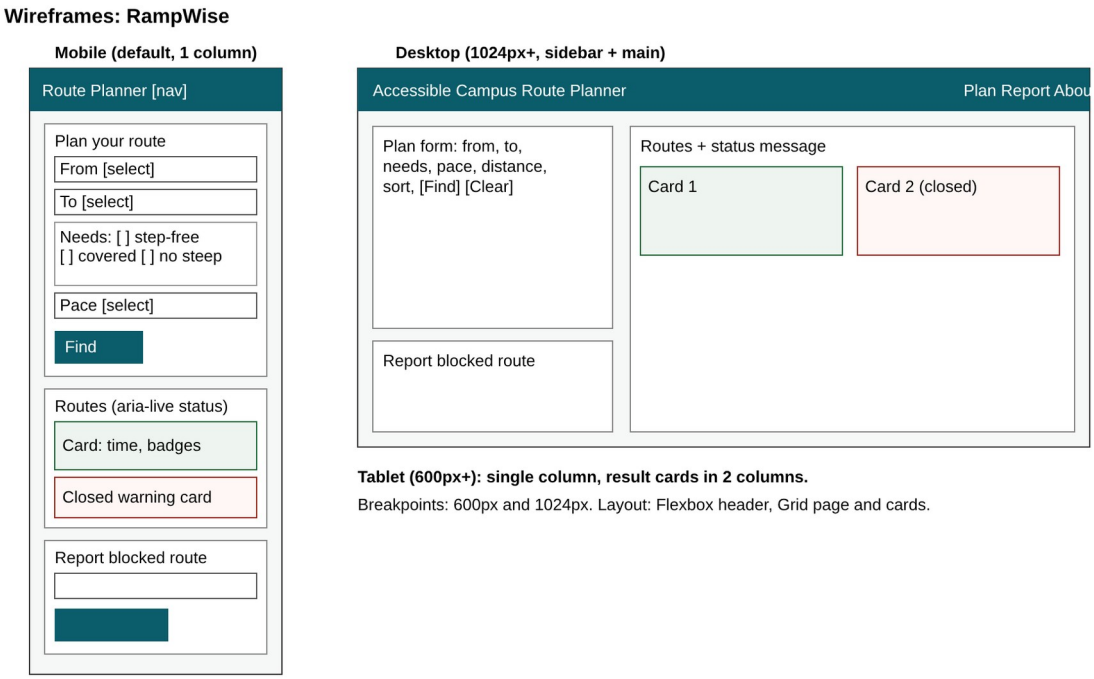
2. WDLC stages

Stage	What I did
Planning	Picked the user and kept the scope small: direct routes between six buildings.
Analysis	Listed what a route needs: distance, steps, slope, lift, cover and open/closed status.
Design	Drew wireframes for mobile and desktop and chose colours and layout (section 4).
Implementation	Wrote the HTML form, then CSS, then the data and JavaScript.
Testing	Tried normal and bad inputs and checked three screen sizes (section 5).
Deployment	Upload to GitHub and publish with GitHub Pages.
Maintenance	Update routes in data.js; later add a server for saving reports.

3. Client and server

At this stage everything runs in the browser. The server only sends the HTML, CSS and JavaScript files. After that, the browser checks the form, filters the routes array, calculates the time and shows the results by itself. Marking a route as closed only changes the array in memory, so it is lost when the page reloads. In Assignments 2 and 3 a server and database would store the routes and reports permanently and share them between users.

4. Wireframe and design



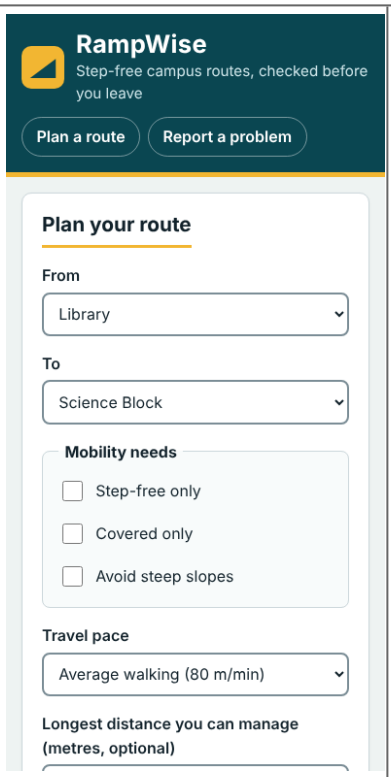
On mobile the page is one column: form, then results, then the report form. At 600px the result cards go into two columns. At 1024px the forms move into a sidebar and the results sit beside them. I used Flexbox for the header and buttons and Grid for the page and cards. For accessibility I added a skip link, labels on every field, a fieldset for the checkboxes, error messages, and aria-live on the results message. Good/bad badges always include text, so colour is not the only signal.

5. Interactions and testing

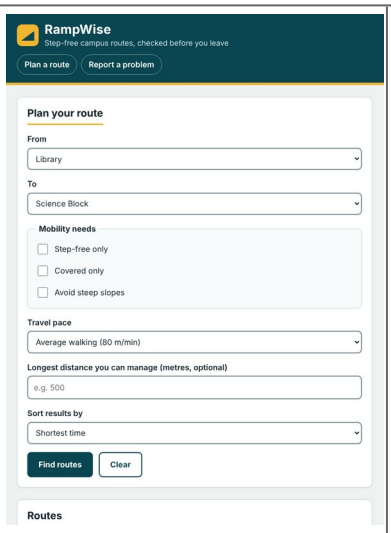
I built three interactions: (1) the route finder filters and sorts the routes array, (2) the time calculator divides distance by the chosen pace and updates as the user changes the form, and (3) the report form marks a route closed and shows a warning on its card. I wrote ten sample routes for six buildings.

Test	Expected result
Library to Science Block	Two routes with time and badges
No building chosen	Message asking to choose start and destination
Same start and destination	Error message
Science Block to Engineering	Closed route shown with a warning
Step-free + covered + max 100 m	No matches message that suggests changing a filter
Report with a very short reason	Error asking for more detail

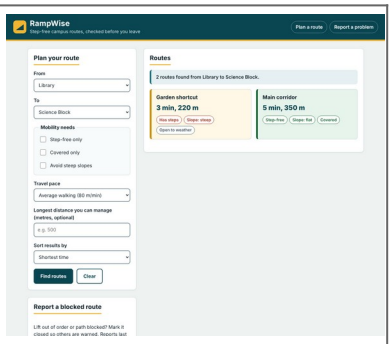
Screenshots of RampWise on mobile, tablet and desktop:



Mobile



Tablet



Desktop

6. Limits and AI use

The app only has direct routes, so it cannot plan a trip with a stop in the middle, and reports are lost on reload.

AI assistance: [state honestly how you used AI, e.g. "An AI assistant generated the first version of the code and this report; I reviewed, tested and edited them."] No frameworks, templates or copied assets were used. All data is made up.